Challenge Strings

0. Challenge

- Challenge hacking android
- Lien du challenge https://www.mobilehackinglab.com/course/lab-strings

1. Analyse Statique et Deep Links

L'application comporte deux activités exportées, identifiables via le fichier `AndroidManifest.xml` :
(Ce fichier est utilisé pour déclarées les permissions d'une application Android)

- `MainActivity` : Lancée par défaut lors de l'ouverture de l'application.
- `Activity2` : Accessible via des **Deep Links** (URI de type `mhl://labs`).

Il est possible d'identifier deux activity exportées, c'est-à-dire accessibles par d'autres applications du système :

Pour rappel, les **deep links** (*Android >= 6.0*) sont des URI conçues pour s'ouvrir directement dans une application mobile, guidant les utilisateurs vers un contenu ou une action spécifique sans passer par l'écran d'accueil de l'application.

Analyse de la logique de sécurité (`Activity2`)

```
Activity2
22  /* JADX INFO: Compiled from: Activity2.kt */
23  /* JADX INFO: loaded from: classes4.dex */
24  @Metadata(d1 = {"\u0000(\n\u0002\u0018\u0002\n\u0002\u0018\u0002\n\u0002\b\u0002\n\u0002\u0010\u000e\n\u0002\b\u0004\n\u0002\u0018\u0002\n\u0002\b\u0002
25  public final class Activity2 extends AppCompatActivity {
26      private final native String getflag();
27
28      @Override // androidx.fragment.app.FragmentActivity, androidx.activity.ComponentActivity, androidx.core.app.ComponentActivity, android.app.Activity
29      protected void onCreate(Bundle savedInstanceState) throws NoSuchPaddingException, NoSuchAlgorithmException {
30          super.onCreate(savedInstanceState);
31          setContentView(R.layout.activity_2);
32          SharedPreferences sharedPreferences = getSharedPreferences("DAD4", 0);
33          String u_1 = sharedPreferences.getString("UUU0133", null);
34          boolean isActionView = Intrinsics.areEqual(getIntent().getAction(), "android.intent.action.VIEW");
35          boolean isU1Matching = Intrinsics.areEqual(u_1, cd());
36          if (isActionView && isU1Matching) {
37              Uri uri = getIntent().getData();
38              if (uri != null && Intrinsics.areEqual(uri.getScheme(), "mhl") && Intrinsics.areEqual(uri.getHost(), "labs")) {
39                  String base64Value = uri.getLastPathSegment();
40                  byte[] decodedValue = Base64.decode(base64Value, 0);
41                  if (decodedValue != null) {
42                      String ds = new String(decodedValue, Charsets.UTF_8);
43                      byte[] bytes = "your_secret_key_1234567890123456".getBytes(Charsets.UTF_8);
44                      Intrinsics.checkNotNullExpressionValue(bytes, "this as java.lang.String).getBytes(charset)");
45                      String str = decrypt("AES/CBC/PKCS5Padding", "bqGrDKdQ8zo26HflRsGvVA==", new SecretKeySpec(bytes, "AES"));
46                      if (str.equals(ds)) {
47                          System.loadLibrary("flag");
48                          String s = getflag();
49                          Toast.makeText(getApplicationContext(), s, 1).show();
50                          return;
51                      } else {
52                          finishAffinity();
53                          finish();
54                          System.exit(0);
55                          return;
56                      }
```

Au sein d'Activity2, on remarque une partie très intéressante ! Il s'agit de **System.loadLibrary(“ flag “)**

En effet, cela permet de charger la librairie native flag (fichier **.so** présent dans le répertoire **lib/**) :

Néanmoins, avant d'arriver à cette déclaration, la fonction **onCreate** ajoute de nombreux contrôles

A. Détail des verrous applicatifs

Pour atteindre l'appel `System.loadLibrary("flag")`, quatre verrous doivent être levés:

1. **Ligne 40 (Vérification de l'Action et Date)** : L'activité doit être lancée par un Intent `VIEW`. La clé `UUU0133` dans `DAD4.xml` doit correspondre exactement à la date système actuelle (`DD/MM/YYYY`) renvoyée par la fonction `cd()`.

```
90      }
91
92      private final String cd() {
93          SimpleDateFormat sdf = new SimpleDateFormat("dd/MM/yyyy", Locale.getDefault());
94          String str = sdf.format(new Date());
95          Intrinsics.checkNotNullExpressionValue(str, "format(...)");
96          Activity2Kt.cu_d = str;
97          String str2 = Activity2Kt.cu_d;
98          if (str2 != null) {
99              return str2;
100         }
101         Intrinsics.throwUninitializedPropertyAccessException("cu_d");
102         return null;
103     }
104 }
```

2. **Ligne 42 (Validation de l'URI)** : L'URI fournie doit utiliser le schéma `mhl` et l'hôte `labs` (ex: `mhl://labs/...`).
3. **Ligne 45 (Segment de chemin)** : L'application extrait le dernier segment de l'URI (`getLastPathSegment()`) et tente de le décoder depuis le Base64.

4. **Ligne 50 (Validation Cryptographique)** : La valeur décodée doit être égale au résultat du déchiffrement AES de la chaîne statique `bqGrDKdQ8zo26HflRsGvVA==` .

B. Paramètres de déchiffrement identifiés:

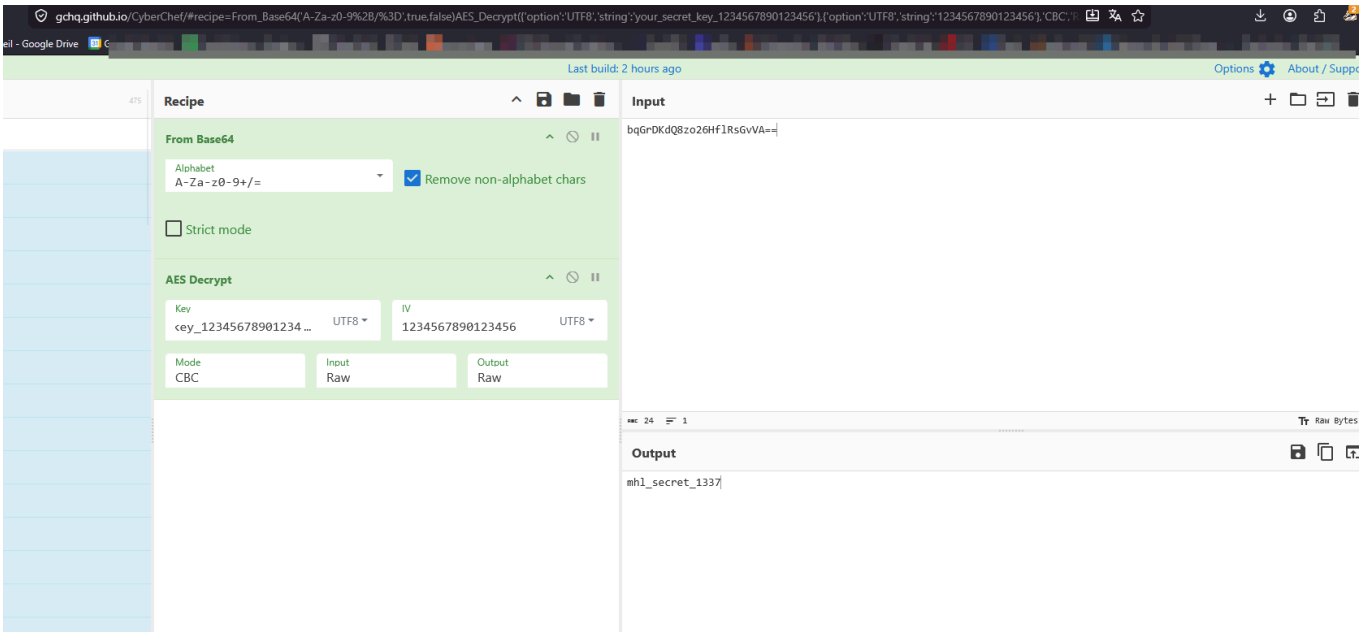
- **Algorithme** : `AES/CBC/PKCS5Padding`
- **Clé (Hardcoded)** : `your_secret_key_1234567890123456`
- **IV (Vecteur d'Initialisation)** : Défini dans `Activity2Kt.fixedIV` comme `1234567890123456` .

```
public final String decrypt(String algorithm, String cipherText, SecretKeySpec key) throws NoSuchPaddingException, NoSuchAlgorith
    Intrinsics.checkNotNullParameter(algorithm, "algorithm");
    Intrinsics.checkNotNullParameter(cipherText, "cipherText");
    Intrinsics.checkNotNullParameter(key, "key");
    Cipher cipher = Cipher.getInstance(algorithm);
    try {
        byte[] bytes = Activity2Kt.fixedIV.getBytes(Charsets.UTF_8);
        Intrinsics.checkNotNullExpressionValue(bytes, "this as java.lang.String).getBytes(charset)");
        IvParameterSpec ivSpec = new IvParameterSpec(bytes);
        cipher.init(2, key, ivSpec);
        byte[] decodedCipherText = Base64.decode(cipherText, 0);
        byte[] decrypted = cipher.doFinal(decodedCipherText);
        Intrinsics.checkNotNull(decrypted);
        return new String(decrypted, Charsets.UTF_8);
    } catch (Exception e) {
        throw new RuntimeException("Decryption failed", e);
    }
}

private final String cd() {
    SimpleDateFormat sdf = new SimpleDateFormat("dd/MM/yyyy", Locale.getDefault());
    String str = sdf.format(new Date());
    Intrinsics.checkNotNullExpressionValue(str, "format(...)");
    Activity2Kt.cu_d = str;
    String str2 = Activity2Kt.cu_d;
    if (str2 != null) {
        return str2;
    }
```

```
1  package com.mobilehackinglab.challenge;
2
3  import androidx.constraintlayout.widget.ConstraintLayout;
4  import kotlin.Metadata;
5
6  /* JADX INFO: compiled from: Activity2.kt */
7  /* JADX INFO: loaded from: classes4.dex */
8  @Metadata(d1 = {"\u0000\n\n\u0000\n\u0002\u0010\u000e\n\u0002\b\u0002\"\u000e\u0010\u0000\u00
9  public final class Activity2Kt {
10     private static String cu_d = null;
11     public static final String fixedIV = "1234567890123456";
12 }
```

- **Chaine à décoder** : `bqGrDKdQ8zo26HflRsGvVA==`

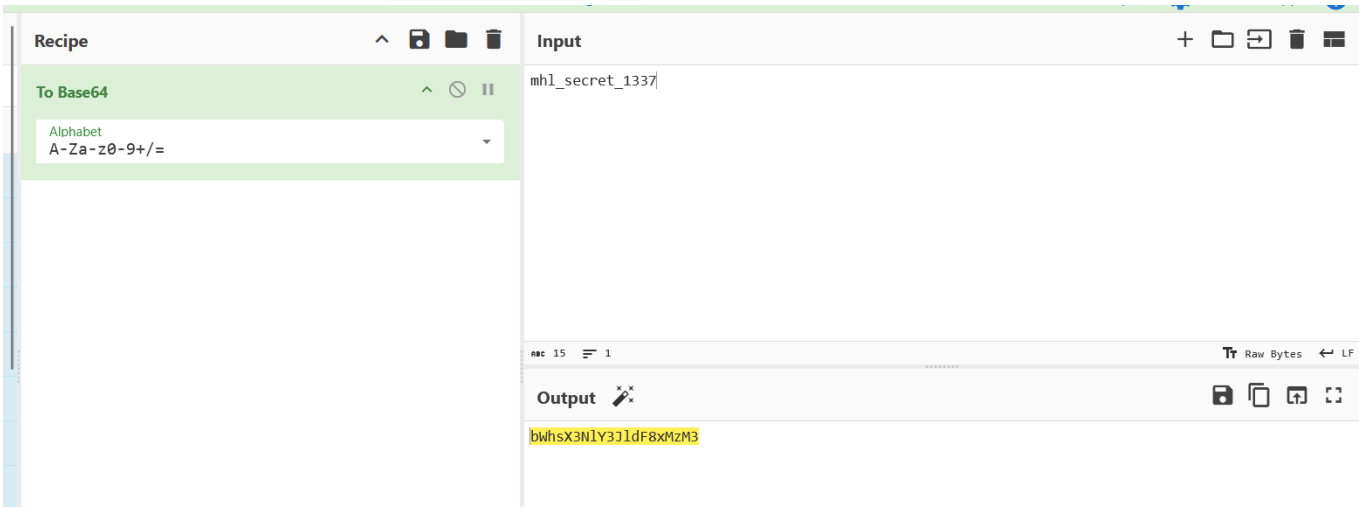


On trouve alors le secret `mhl_secret_1337`

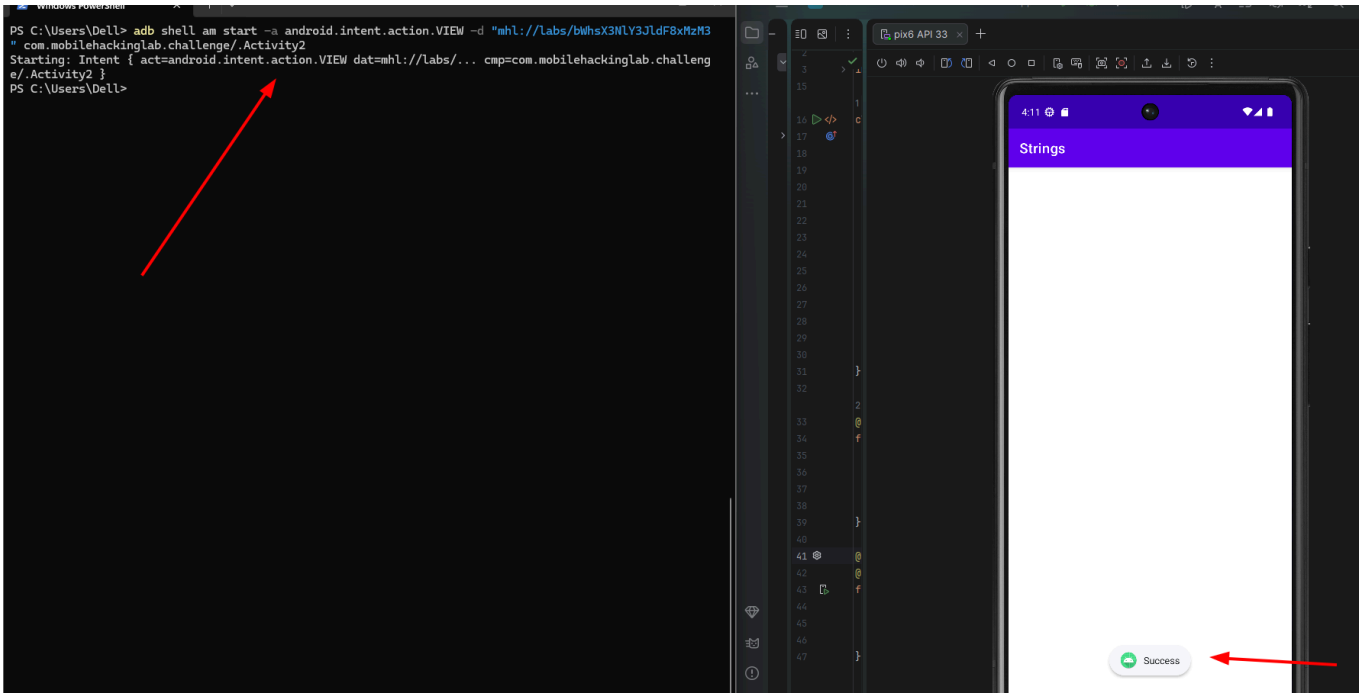
Tests du Intent

L'**Intent** est un concept fondamental du développement Android. Il s'agit d'un objet de messagerie utilisé pour demander une action à un autre composant d'application (comme une Activité, un Service ou un Broadcast Receiver).

Pour cela on doit encodée `mhl_secret_1337`




```
adb shell am start -a android.intent.action.VIEW -d
"mhl://labs/bWhsX3NlY3JldF8xMzM3" com.mobilehackinglab.challenge/.Activity2
```



3. Bypass du contrôle temporel (Shared Preferences)

Le premier obstacle majeur est la vérification de la date via la clé `UUU0133` stockée dans les préférences de l'application. Comme l'application utilise `System.exit(0)` si cette donnée est absente ou incorrecte, nous devons injecter ce fichier manuellement.

A. Accès au stockage privé

L'application stocke ses préférences dans `/data/data/com.mobilehackinglab.challenge/shared_prefs/`. Ce répertoire est protégé par le système Android et n'est accessible qu'avec des privilèges **Root**.

B. Commandes d'injection

En utilisant `adb shell`, nous avons exécuté la suite de commandes suivante pour forger l'environnement nécessaire

```
PS C:\Users\Dell\Downloads\com.mobilehackinglab.strings> adb shell
emu64x:/ $ su
emu64x:/ # mkdir -p /data/data/com.mobilehackinglab.challenge/shared_prefs
emu64x:/ # echo '<?xml version="1.0" encoding="utf-8" standalone="yes"?>
> <map>
>     <string name="UUU0133">13/04/2026</string>
> </map>' > /data/data/com.mobilehackinglab.challenge/shared_prefs/DAD4.xml
emu64x:/ #  chmod -R 777 /data/data/com.mobilehackinglab.challenge/shared_prefs/
emu64x:/ # chown -R system:system /data/data/com.mobilehackinglab.challenge/shared_prefs/
emu64x:/ #
emu64x:/ #
emu64x:/ # cat /data/data/com.mobilehackinglab.challenge/shared_prefs/DAD4.xml
<?xml version="1.0" encoding="utf-8" standalone="yes"?>
<map>
    <string name="UUU0133">13/04/2026</string>
</map>
emu64x:/ # exit
emu64x:/ $ exit
PS C:\Users\Dell\Downloads\com.mobilehackinglab.strings> date

lundi 13 avril 2026 18:38:46


PS C:\Users\Dell\Downloads\com.mobilehackinglab.strings>
```

Le fichier `DAD4.xml` est un fichier de préférences Android (**SharedPreferences**) utilisé par l'application pour stocker et vérifier une condition temporelle (la date du jour sous la clé `UUU0133`) indispensable au lancement de l'activité protégée.

5. Résultat Final

On veut pouvoir récupérer le flag proprement maintenant !

A Configuration de frida-server

Le **frida-server** est un binaire exécuté sur l'appareil cible (Android) qui permet à l'outil Frida sur mon ordinateur de s'y connecter pour injecter des scripts et manipuler les processus en temps réel.

```
adb push .\frida-server-17.9.1-android-x86_64\frida-server-17.9.1-android-x86_64 /data/local/tmp/frida-server  # On dépose le binaire
setenforce 0 #Désactive temporairement les restrictions de sécurité de SELinux
/data/local/tmp/frida-server & # Lancement de frida-server pour la comunication
```

B. Préparation du script

Le script JavaScript est utilisé avec Frida pour **instrumenter dynamiquement** l'application en temps réel, permettant d'intercepter l'appel à la fonction native et d'extraire le flag directement depuis la mémoire vive.

Voici le script qu'on va injecter

```
Java.perform(function () {
    const Activity2 = Java.use('com.mobilehackinglab.challenge.Activity2');

    Activity2.getflag.implementation = function () {
        // 1. On appelle la fonction originale
        const result = this.getflag();

        // 2. On va chercher la bibliothèque en mémoire
        const m = Process.findModuleByName("libflag.so");

        // 3. On scanne la mémoire de cette bibliothèque pour trouver "MHL{"
        // (Le flag est chargé en mémoire dès que la lib est initialisée)
        Memory.scan(m.base, m.size, "4d 48 4c 7b", { // 4d 48 4c 7b = MHL{
            onMatch: function (address, size) {
                console.log("\n========================================");
                console.log(" FLAG TROUVÉ : " + address.readUtf8String());
                console.log("========================================\n");
            },
            onComplete: function () {}
        });

        return result;
    };
});
```

C. Lancement du script

Puis on relance l'Intent et on trouve le flag

```
PS C:\Users\Dell> adb shell am start -a android.intent.action.VIEW -d "mhl://labs/bWhsX3NlY3JldF8xMzM3
" com.mobilehackinglab.challenge/.Activity2
Starting: Intent { act=android.intent.action.VIEW dat=mhl://labs/... cmp=com.mobilehackinglab.challeng
e/.Activity2 }
PS C:\Users\Dell>
```

```
PS C:\Users\Dell\Downloads\com.mobilehackinglab.strings> frida -U -f com.mobilehackinglab.challenge -
 scan_flag.js
     ____
    / _  |   Frida 17.9.1 - A world-class dynamic instrumentation toolkit
   | (_| |
    > _  |   Commands:
   /_/ |_|       help      -> Displays the help system
   . . . .       object?   -> Display information about 'object'
   . . . .       exit/quit -> Exit
   . . . .
   . . . .   More info at https://frida.re/docs/home/
   . . . .
   . . . .   Connected to Android Emulator 5554 (id=emulator-5554)
Spawned `com.mobilehackinglab.challenge`. Resuming main thread!
[Android Emulator 5554::com.mobilehackinglab.challenge ]->
========================================
 FLAG TROUVÉ : MHL{IN_THE_MEMORY}
========================================
```